📊 AWS Observability & Monitoring — Category 7: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** CloudWatch Metrics, Logs, Alarms, Dashboards, X-Ray, EMF, CloudWatch Agent, ADOT, Container Insights, Lambda Insights

---

📋 Table of Contents

1. 7.1 CloudWatch Metrics — Namespaces, Dimensions, Resolution
2. 7.2 CloudWatch Logs — Log Groups, Metric Filters, Insights
3. 7.3 CloudWatch Alarms — States, Actions, Composite Alarms
4. 7.4 CloudWatch Dashboards & Contributor Insights
5. 7.5 AWS X-Ray — Tracing, Segments, Sampling
6. 7.6 CloudWatch Embedded Metric Format (EMF)
7. 7.7 CloudWatch Agent — Custom Metrics, Procstat
8. 7.8 AWS Distro for OpenTelemetry (ADOT)
9. 7.9 Container Insights & Lambda Insights

7.1 CloudWatch Metrics — Namespaces, Dimensions, Resolution

🟢 What It Is in Simple Terms

CloudWatch Metrics is AWS's time-series data store for numerical measurements about your AWS resources and applications. Every number that describes how your system is behaving — CPU utilization, request count, latency, error rate — is a metric. CloudWatch collects, stores, and lets you query, visualize, and alert on these numbers.

🔍 Why It Exists / What Problem It Solves

Without a centralized metrics system, you'd be SSH-ing into individual servers to run `top` or `vmstat`, checking each service's dashboard manually, and having no historical data to compare against. CloudWatch gives you a single pane of glass for all your AWS resources with automatic metric collection, long-term retention, and alerting.

⚙️ How It Works Internally

```
CloudWatch Metrics Architecture:

Data Point:
├── Namespace:   "AWS/EC2"
├── MetricName:  "CPUUtilization"
├── Dimensions:  {InstanceId: "i-0abc123"}
├── Value:       72.5
├── Unit:        "Percent"
└── Timestamp:   2024-01-15T14:30:00Z


┌──────────────────────────────────────────────────────────┐
│              CloudWatch Metrics Store                    │
│                                                          │
│  Namespace: AWS/EC2                                      │
│  ├── CPUUtilization                                      │
│  │   ├── {InstanceId: i-0abc123} → [72.5, 68.1, 71]      │
│  │   └── {InstanceId: i-0def456} → [45.2, 44.8, 46]      │
│  ├── NetworkIn                                           │
│  │   └── {InstanceId: i-0abc123} → [...]                 │
│  └── DiskReadOps                                         │
│      └── ...                                             │
│                                                          │
│  Namespace: MyApp/Orders                                 │
│  └── OrdersProcessed                                     │
│      └── {Environment: prod, Region: us-east-1}          │
└──────────────────────────────────────────────────────────┘
```

🧩 Namespaces

```
Namespace = container/category for metrics
           Prevents naming collisions between AWS services and your app

AWS built-in namespaces:
├── AWS/EC2         → CPUUtilization, NetworkIn, StatusCheckFailed
├── AWS/RDS         → DatabaseConnections, FreeStorageSpace, ReadLatency
├── AWS/Lambda      → Invocations, Duration, Errors, Throttles,
ConcurrentExecutions
├── AWS/ECS         → CPUUtilization, MemoryUtilization
(cluster/service/task)
├── AWS/ELB         → RequestCount, Latency, HTTPCode_ELB_4XX_Count
├── AWS/ALB         → RequestCount, TargetResponseTime,
HTTPCode_Target_5XX_Count
├── AWS/SQS         → NumberOfMessagesSent,
ApproximateNumberOfMessagesVisible
├── AWS/DynamoDB    → ConsumedReadCapacityUnits, SystemErrors,
SuccessfulRequestLatency
└── AWS/S3          → BucketSizeBytes, NumberOfObjects (daily — not real-
time!)
```

```
Custom namespaces (your own app metrics):
├── MyApp/Orders    → OrdersProcessed, OrdersFailedValidation
├── MyApp/API       → RequestLatencyP99, ErrorRate
└── MyService/Cache → CacheHitRatio, CacheMissCount
```

🧩 Dimensions

```
Dimensions = key-value pairs that identify a specific metric
             They are the labels/filters for a metric

Without dimensions: "CPUUtilization" → which EC2 instance?
With dimensions:    "CPUUtilization" {InstanceId: "i-0abc123"} → specific!

Dimensions enable filtering and aggregation:

Query: CPUUtilization for instance i-0abc123
Dimensions: {InstanceId: "i-0abc123"}

Query: Average CPUUtilization for AutoScaling group "web-asg"
Dimensions: {AutoScalingGroupName: "web-asg"}

Query: Total RequestCount for ALB "prod-alb"
Dimensions: {LoadBalancer: "app/prod-alb/xyz123"}

⚠ Dimensions are part of the metric identity.
   "CPUUtilization" {InstanceId: "i-abc"} and
   "CPUUtilization" {InstanceId: "i-def"} are TWO different metrics.

Max dimensions per metric: 30
```

```bash
# Publishing custom metrics via CLI
aws cloudwatch put-metric-data \
  --namespace "MyApp/Orders" \
  --metric-data '[{
    "MetricName": "OrdersProcessed",
    "Dimensions": [
      {"Name": "Environment", "Value": "prod"},
      {"Name": "Region",      "Value": "us-east-1"}
    ],
    "Value": 142,
    "Unit": "Count",
    "Timestamp": "2024-01-15T14:30:00Z"
  }]'
```

🧩 Resolution

```
Two types of metric resolution:

Standard Resolution (default):
├── Data point stored every 1 MINUTE
├── Free for most AWS service metrics
├── Retention:
│   ├── 1 minute:  15 days
│   ├── 5 minutes: 63 days
│   └── 1 hour:    455 days (15 months — auto-downsampled)
└── Alarms: minimum evaluation period = 1 minute

High Resolution:
├── Data point stored every 1-60 SECONDS
├── Costs: $0.30 per custom metric per month
├── Retention:
│   ├── 1 second:  3 hours
│   └── 1 minute:  15 days (then same as standard)
└── Alarms: evaluation period as low as 10 seconds

Use high resolution when:
├── Detecting sub-minute spikes (Lambda throttles, latency spikes)
├── Tracking rapidly changing metrics (error rates, TPS)
└── Gaming / real-time applications
```

```aws
# Publishing high-resolution metric
aws cloudwatch put-metric-data \
  --namespace "MyApp/API" \
  --metric-data '[{
    "MetricName": "Latency",
    "Value": 145,
    "Unit": "Milliseconds",
    "StorageResolution": 1
  }]'
  # Default StorageResolution: 60 (standard)

# Get metric statistics with percentiles
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda \
  --metric-name Duration \
  --dimensions Name=FunctionName,Value=my-function \
  --start-time 2024-01-15T00:00:00Z \
  --end-time   2024-01-15T23:59:59Z \
  --period 300 \
  --extended-statistics p50 p90 p99

# Get multiple metrics at once (more powerful)
aws cloudwatch get-metric-data \
  --metric-data-queries '[{
    "Id": "cpu",
    "MetricStat": {
```

```
        "Metric": {
          "Namespace": "AWS/EC2",
          "MetricName": "CPUUtilization",
          "Dimensions": [{"Name": "InstanceId", "Value": "i-0abc123"}]
        },
        "Period": 300,
        "Stat": "Average"
      }
    }]' \
    --start-time 2024-01-15T00:00:00Z \
    --end-time   2024-01-15T23:59:59Z
```

```
Statistics (aggregations over a period):
├── Average:     mean of all data points in period
├── Sum:         total of all data points
├── Minimum:     lowest data point
├── Maximum:     highest data point
├── SampleCount: number of data points
└── Percentiles: p50, p90, p99, p99.9 (critical for latency metrics)
```

💬 Short Crisp Interview Answer

"CloudWatch Metrics is AWS's time-series database for numerical monitoring data. Every metric is identified by its namespace (the category like AWS/EC2 or your custom MyApp/API), metric name, and dimensions (key-value pairs that filter to a specific resource like InstanceId=i-abc123). Standard resolution stores data every minute with automatic downsampling over time — 1-minute data kept 15 days, then aggregated. High resolution stores every 1-60 seconds for detecting sub-minute spikes. The key operational detail: dimensions are part of the metric identity, so adding or removing a dimension creates a completely different metric in CloudWatch's storage."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Dimensions change = different metric	Adding a dimension to an existing metric creates a new time series — old data is separate
S3 metrics are daily	BucketSizeBytes and NumberOfObjects only updated once per day — not real-time
High-res retention is short	1-second data kept only 3 hours. Zoom out quickly during incidents
Metric math period alignment	Math expressions require consistent periods. Mixing 60s and 300s causes issues
Custom metric cost	Each unique namespace + metric name + dimension combo = $0.30/month

---

7.2 CloudWatch Logs — Log Groups, Metric Filters, Insights

🟢 What It Is in Simple Terms

CloudWatch Logs is AWS's centralized log management service. Applications, AWS services, and OS-level agents all ship logs here. You can store, search, filter, and analyze log data in real time and retroactively.

⚙️ How It Works Internally

```
CloudWatch Logs Structure:

Log Group
└── Log Stream 1 (one Lambda execution env, one EC2 instance)
    ├── Log Event 1: {timestamp, message}
    ├── Log Event 2: {timestamp, message}
    └── Log Event 3: {timestamp, message}
└── Log Stream 2
    └── ...

Log Group = logical grouping of log streams from one source type
  Examples:
  /aws/lambda/my-function      → one log stream per Lambda execution env
  /aws/ecs/prod/api-service    → one log stream per ECS task
  /var/log/messages            → one log stream per EC2 instance
  /aws/rds/instance/mydb/error → RDS error logs

Log Stream = sequence of log events from ONE source
  (one Lambda execution environment, one EC2 instance)

Log Event = individual log entry
  {
    "timestamp": 1705329000000,  // milliseconds epoch
    "message": "[ERROR] Connection timeout to db.prod:5432"
  }
```

🧩 Log Groups — Configuration

```
Log Group settings:
├── Retention: 1 day to 10 years (or Never Expire)
│   Default: Never Expire — you pay forever!
├── KMS encryption: encrypt logs at rest with KMS key
```

```
├── Metric filters: create metrics FROM log content
└── Subscription filters: stream logs to Lambda, Kinesis, OpenSearch
```

```aws
# Setting retention policy — ALWAYS do this!
aws logs put-retention-policy \
  --log-group-name /aws/lambda/my-function \
  --retention-in-days 30

# Create log group with retention in one workflow
aws logs create-log-group \
  --log-group-name /myapp/api \
  --tags Environment=prod

aws logs put-retention-policy \
  --log-group-name /myapp/api \
  --retention-in-days 90
```

```
⚠ Default retention is NEVER EXPIRE.
   Every log group MUST have a retention policy set.
   Otherwise you pay forever for logs you'll never look at.
```

🧩 Metric Filters

```
Metric Filters = extract numbers FROM log text → create CloudWatch Metrics

Use case: log contains error messages → create error rate metric
          log contains latency values → create latency metric

Filter pattern syntax:
├── Simple text match:  "ERROR"
├── JSON field:         { $.statusCode = 500 }
├── Space-delimited:    [host, ident, auth, timestamp, request, status=5*,
size]
└── Multiple terms:     "ERROR" "Connection refused"
```

```aws
# Count 5XX errors from nginx access log
# Log line: 10.0.1.5 - - [15/Jan/2024] "GET /api/users HTTP/1.1" 500 234
aws logs put-metric-filter \
  --log-group-name /aws/alb/access-logs \
  --filter-name "HTTP5xxErrors" \
  --filter-pattern '[host, ident, auth, timestamp, request,
status_code=5*, size]' \
  --metric-transformations '[{
```

```
    "metricName": "HTTP5xxCount",
    "metricNamespace": "MyApp/ALB",
    "metricValue": "1",
    "unit": "Count"
  }]'

# Extract latency value from JSON log
# Log line:
{"level":"INFO","requestId":"abc123","latency":145,"path":"/api/orders"}
aws logs put-metric-filter \
  --log-group-name /myapp/api \
  --filter-name "APILatency" \
  --filter-pattern '{ $.level = "INFO" && $.latency > 0 }' \
  --metric-transformations '[{
    "metricName": "RequestLatency",
    "metricNamespace": "MyApp/API",
    "metricValue": "$.latency",
    "unit": "Milliseconds"
  }]'
```

```
⚠ Metric filters only process FUTURE log events after creation.
   They do NOT backfill historical data.
   Metric filters can emit COUNT or a value from the log line.
   Percentiles require custom EMF or SDK metric publishing.
```

🧩 CloudWatch Logs Insights

```
CloudWatch Logs Insights:
├── SQL-like query language for log data
├── Interactive, fast queries across log groups
├── Supports aggregations, sorting, visualization
└── Designed for ad-hoc investigation during incidents

Key commands:
├── fields:  select specific fields to display
├── filter:  where clause (like SQL WHERE)
├── sort:    order results
├── limit:   limit returned rows
├── stats:   aggregate functions (count, avg, sum, min, max, percentile)
├── parse:   extract fields from unstructured text using patterns
└── dedup:   deduplicate results by a field
```

```
# Count errors by type
fields @timestamp, errorType
| filter level = "ERROR"
| stats count(*) as errorCount by errorType
```

```
| sort errorCount desc

# P99 latency per API endpoint
fields @timestamp, path, latency
| filter ispresent(latency)
| stats pct(latency, 99) as p99, count(*) as requests by path
| sort p99 desc

# Find slow Lambda cold starts
fields @timestamp, @duration, @initDuration
| filter ispresent(@initDuration)
| stats avg(@initDuration) as avgColdStart, max(@initDuration) as
maxColdStart
| sort maxColdStart desc

# Parse unstructured logs
fields @message
| parse @message "user=* action=* duration=*ms" as user, action, duration
| stats avg(duration) by action

# Error rate over time (for chart visualization)
fields @timestamp
| filter level = "ERROR"
| stats count(*) as errors by bin(5m)
```

```
# Query multiple log groups simultaneously
aws logs start-query \
  --log-group-names /myapp/api /myapp/worker \
  --start-time $(date -d '1 hour ago' +%s) \
  --end-time $(date +%s) \
  --query-string 'fields @timestamp, @message | filter level = "ERROR" |
limit 100'
```

```
Query limits:
├── Up to 50 log groups per query
├── Results limited to 10,000 rows
├── Maximum query duration: 15 minutes
└── Cost: $0.005 per GB of data scanned
```

🧩 Log Subscriptions — Streaming Logs Out

```
Subscription Filters stream real-time log data to other services:
├── Lambda:           real-time processing (parse, transform, alert)
├── Kinesis Firehose: buffer and deliver to S3, Redshift, OpenSearch
└── Kinesis Data Streams: custom real-time processing
```

```
Use case — ship logs to OpenSearch:
CW Logs → Subscription Filter → Kinesis Firehose → OpenSearch

Use case — real-time alerting on specific log patterns:
CW Logs → Subscription Filter → Lambda → SNS → PagerDuty

Cross-account log aggregation:
Account A (prod): CW Logs → Subscription → Kinesis → Account B (logging)
Account B:        Central OpenSearch cluster for all accounts
```

```
aws logs put-subscription-filter \
  --log-group-name /myapp/api \
  --filter-name "AllLogs" \
  --filter-pattern "" \
  --destination-arn arn:aws:firehose:...:deliverystream/logs-to-opensearch
```

💬 Short Crisp Interview Answer

"CloudWatch Logs organizes log data in a hierarchy: log groups (one per application/service), log streams (one per instance/execution environment), and log events (individual lines). The most important operational concern is setting retention policies — the default is Never Expire and you pay for all stored data indefinitely. Metric Filters extract numerical metrics from log text using filter patterns — they create CloudWatch metrics from log content but only process future events, not historical. CloudWatch Logs Insights provides an interactive SQL-like query language for ad-hoc log analysis — great for incident investigation with aggregations like P99 latency per endpoint or error counts by type. Subscription filters stream logs in real time to Lambda, Kinesis Firehose, or Kinesis Streams for further processing."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Default retention = Never Expire	Set retention on every log group immediately — costs accumulate forever
Metric filters don't backfill	Only process new log events after filter creation
Insights cost	$0.005/GB scanned. Querying a busy service for 30 days can be expensive
Log stream per execution env	Each Lambda cold start creates a new log stream — hundreds of streams per function
Subscription filter limit	Only 2 subscription filters per log group
Insights 10K row limit	Results truncated at 10,000 rows — use aggregations, not raw log pulls

---

7.3 CloudWatch Alarms — States, Actions, Composite Alarms

🟢 What It Is in Simple Terms

CloudWatch Alarms watch a metric and take actions when it crosses a threshold. They're the bridge between metrics and operations — from "I see a high error rate" to "someone gets paged and auto-scaling kicks in."

⚙️ Alarm States

```
Three alarm states:

OK:                   Metric is within acceptable range
                      (below threshold or within expected bounds)

ALARM:                Metric has breached the threshold for the
                      configured number of evaluation periods.
                      Actions triggered: SNS, Auto Scaling, EC2 actions

INSUFFICIENT_DATA:    Not enough data points to evaluate the alarm.
                      Occurs when:
                      - Alarm just created (no data yet)
                      - Metric stopped being published
                      - High-resolution alarm waiting for data points

State transitions trigger actions:
OK → ALARM:                page on-call, scale out, stop instance
ALARM → OK:                resolve page, scale in
ALARM → INSUFFICIENT_DATA: alert that monitoring broke
```

🧩 Alarm Configuration

```
Alarm anatomy:
┌──────────────────────────────────────────────────────────┐
│ Alarm: HighCPU-prod-web                                  │
│                                                          │
│ Metric:     AWS/EC2 CPUUtilization                       │
│ Dimension:  AutoScalingGroupName = prod-web-asg          │
│ Statistic:  Average                                      │
│ Period:     300 seconds (5 minutes)                      │
│                                                          │
│ Condition:  CPUUtilization > 80 (Threshold)              │
│                                                          │
```

```
│ Evaluation: 3 out of 3 datapoints breaching                │
│             (must be above 80 for 15 minutes to alarm)     │
│             (prevents noise from momentary spikes)         │
│                                                            │
│ Missing data: treat as breaching / not breaching           │
│                                                            │
│ Actions:                                                   │
│   On ALARM: SNS → PagerDuty, Auto Scaling scale out        │
│   On OK:    SNS → resolve PagerDuty                        │
└────────────────────────────────────────────────────────────┘
```

```
aws cloudwatch put-metric-alarm \
  --alarm-name "HighCPU-prod-web" \
  --alarm-description "CPU > 80% for 15 minutes" \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=AutoScalingGroupName,Value=prod-web-asg \
  --statistic Average \
  --period 300 \
  --evaluation-periods 3 \
  --datapoints-to-alarm 3 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:...:prod-alerts \
  --ok-actions arn:aws:sns:...:prod-alerts
```

```
Datapoints to alarm (M out of N):
├── evaluationPeriods = N (total periods to consider)
├── datapointsToAlarm = M (how many must breach)
├── 3/3 = alarm only if ALL 3 consecutive periods breach (low noise)
└── 2/3 = alarm if ANY 2 of 3 periods breach (faster detection)
    Good for flapping: avoids false alarms from brief spikes

Missing data treatment:
├── breaching:    treat missing data as threshold exceeded (safe default)
├── notBreaching: treat missing data as OK (system default)
├── ignore:       maintain current alarm state
└── missing:      transition to INSUFFICIENT_DATA state

⚠ For metrics that should ALWAYS have data (Lambda Errors,
   API Gateway requests): use "breaching".
   Missing data often means the service stopped working entirely!
```

🧩 Alarm Actions

```
Actions CloudWatch Alarms can trigger:

1. SNS Topic → notifications everywhere:
   SNS → Email
   SNS → SMS
   SNS → Lambda (custom logic)
   SNS → PagerDuty / OpsGenie / Slack (via HTTPS endpoint)
   SNS → SQS (queue-based remediation)

2. Auto Scaling:
   Scale out: add EC2 instances when CPU > 80%
   Scale in:  remove EC2 instances when CPU < 20%
   (ASG scaling policies use alarm triggers directly)

3. EC2 Actions (for specific instance alarms):
   Stop instance:     when no CPU activity detected
   Terminate instance: when instance is unhealthy
   Reboot instance:   on status check failure
   Recover instance:  move to different hardware on hardware failure

   EC2 Instance Recovery = moves instance to new host while keeping:
   same instance ID, Elastic IP, private IP, instance metadata

4. Systems Manager OpsItem:
   Create operational work item for incident tracking

5. Lambda:
   Direct Lambda invocation via SNS subscription
   For custom automated remediation workflows
```

🧩 Composite Alarms

```
Problem with many individual alarms:
├── Too many alarms → alert fatigue
├── High CPU + High Memory may not be a problem individually,
│   but together = definitely a problem
├── Can't express "alert only if BOTH conditions are true"
└── Can't suppress alarms during maintenance windows

Composite Alarms = logical combination of alarms
                   using AND / OR / NOT operators
```

```bash
# Only alert if CPU is high AND memory is high
aws cloudwatch put-composite-alarm \
  --alarm-name "HighResourcePressure-prod" \
  --alarm-rule "ALARM(HighCPU-prod) AND ALARM(HighMemory-prod)" \
  --alarm-actions arn:aws:sns:...:prod-critical
```

```
# Alert if ANY critical component is in ALARM state
aws cloudwatch put-composite-alarm \
  --alarm-name "ServiceDegraded-prod" \
  --alarm-rule "ALARM(DB-HighLatency) OR ALARM(API-5xxErrors) OR
ALARM(Queue-Depth-Critical)"

# Suppress alert during maintenance window
# (create an alarm that's always in ALARM during maintenance)
# Then: ALARM(ServiceDegraded) AND NOT ALARM(MaintenanceWindow)
```

```
Composite alarm benefits:
├── Reduce alert noise (only meaningful conditions trigger pages)
├── Complex conditions (AND/OR/NOT)
├── Suppress alarms during planned maintenance
└── Single alarm for SLA monitoring combining multiple signals

⚠ Composite alarms cannot directly trigger Auto Scaling actions.
   Route through SNS → Lambda → Auto Scaling logic for automation.
   Auto Scaling actions must be attached to individual metric alarms.
```

💬 Short Crisp Interview Answer

"CloudWatch Alarms have three states: OK, ALARM, and INSUFFICIENT_DATA. When a metric breaches a threshold for M out of N evaluation periods, the alarm transitions to ALARM and triggers actions — SNS notifications, Auto Scaling policies, or EC2 actions like instance recovery. The M-of-N pattern (e.g., 2 out of 3 periods) prevents noise from brief spikes. Missing data treatment is critical: use 'breaching' for metrics that should always have data — if your API stops generating request metrics, that usually means it stopped working. Composite alarms combine multiple individual alarms with AND/OR/NOT logic, letting you page only when truly meaningful conditions occur and suppressing alert fatigue from correlated signals."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Missing data = notBreaching by default	Service down → no metrics → alarm stays OK. Use 'breaching' for critical services
Period × evaluationPeriods = alert delay	300s × 3 periods = 15-minute delay before alarm. Tune for your SLA requirements
Composite alarms cost more	$0.50/month vs $0.10/month for metric alarms
Auto Scaling on composite alarms	Not directly supported — route through SNS → Lambda

Gotcha	Detail
Alarm actions fire on transition only	Action fires once on OK→ALARM transition, not on every evaluation period

- **3/3 consecutive periods misses short sharp incidents** — the most common CloudWatch alarm misconfiguration. Use 2/3 (M of N) to catch incidents that don't sustain for the full evaluation window. Always ask: "what is the shortest incident we must catch?" and work backwards to set period × datapoints-to-alarm.
- **Period × evaluation-periods = detection delay** — a 5-minute period with 3/3 means minimum 15 minutes before alarm fires. For SLA-critical services, reduce period to 60 seconds and use high-resolution metrics for sub-minute detection.

7.4 CloudWatch Dashboards & Contributor Insights

🟢 What It Is in Simple Terms

Dashboards give you a visual, real-time view of your metrics in one place. Contributor Insights automatically identifies which specific dimension values (which user, which IP, which resource) are contributing the most to a metric or log pattern — answering "who is causing this problem?"

🧩 CloudWatch Dashboards

```
Dashboard widget types:
├── Line chart:    metric over time (CPU, latency trends)
├── Number:        current value or aggregate (total requests today)
├── Gauge:         percentage fill visualization (disk usage)
├── Bar chart:     compare values across dimensions
├── Text/Markdown: descriptions, runbooks, links
├── Alarm status:  current state of multiple alarms at a glance
└── Log table:     recent log events from a Logs Insights query

Dashboard best practices:
├── One dashboard per service / domain
├── Top row:    health indicators (alarm statuses, error rates)
├── Middle rows: key performance metrics (latency, throughput)
└── Bottom rows: infrastructure metrics (CPU, memory, disk)
```

```
aws cloudwatch put-dashboard \
  --dashboard-name "prod-api-health" \
  --dashboard-body '{
    "widgets": [{
      "type": "metric",
```

```
      "x": 0, "y": 0, "width": 12, "height": 6,
      "properties": {
        "title": "API Latency P99",
        "metrics": [
          ["MyApp/API", "RequestLatency", "Environment", "prod",
           {"stat": "p99", "period": 60, "label": "P99 Latency"}]
        ],
        "period": 60,
        "view": "timeSeries",
        "yAxis": {"left": {"min": 0, "max": 2000, "label": "ms"}}
      }
    }]
  }'
```

```
Dashboard features:
├── Cross-region:     add metrics from multiple regions in one view
├── Cross-account:    add metrics from other AWS accounts (with sharing)
├── Auto refresh:     set to 10s/1m/2m/5m/15m
├── Math expressions: calculate derived metrics inline
└── Dark mode:        easy on the eyes during incident response at 3am

Cost: $3/dashboard/month (first 3 dashboards are free)
```

🧩 Metric Math

```
Metric Math computes new time series FROM existing metrics:

Example: Error rate = (5xx errors / total requests) × 100

metrics:
  - id: m1
    metric: AWS/ALB HTTPCode_Target_5XX_Count SUM period:60
  - id: m2
    metric: AWS/ALB RequestCount SUM period:60
  - id: errorRate
    expression: (m1 / m2) * 100
    label: "Error Rate %"

Available functions:
├── ANOMALY_DETECTION_BAND(metric, stdDev) → anomaly upper/lower bounds
├── FILL(metric, value) → fill gaps in missing data
├── IF(condition, trueValue, falseValue)
├── SUM([m1, m2, m3]) → sum multiple metrics into one
├── METRICS('prefix') → all metrics whose ID matches prefix
└── RUNNING_SUM → running cumulative total

Anomaly Detection:
├── ML model learns normal behavior pattern for a metric
```

```
├── ANOMALY_DETECTION_BAND creates expected range around normal
├── Alarm when metric goes OUTSIDE the band (vs fixed threshold)
└── Automatically adjusts for: day of week, time of day, seasons
    ⚠ Needs ~2 weeks of data to learn seasonal patterns reliably
```

🧩 Contributor Insights

```
Contributor Insights:
Problem: 10,000 users hit your API. Error rate spikes to 5%.
         Which specific users or IPs are generating the errors?
         Which DynamoDB partition keys are being throttled?

Without Contributor Insights:
→ Write custom aggregation code
→ Parse logs manually in Logs Insights repeatedly
→ Time-consuming during an active incident

With Contributor Insights:
→ Define a rule pointing at a log group
→ AWS automatically ranks top-N contributors in real time
→ Leaderboard: "these 10 users cause 80% of all errors right now"

Use cases:
├── Find DynamoDB hot partition keys (which PK has most throttles)
├── Find top IP addresses hitting your API (DDoS detection)
├── Find top error-generating user IDs
├── Find slowest API endpoints by response time
└── Find which Lambda functions consume most concurrency
```

```
aws cloudwatch put-insight-rule \
  --rule-name "TopErrorUsers" \
  --rule-state ENABLED \
  --rule-definition '{
    "Schema": {"Name": "CloudWatchLogRule", "Version": 1},
    "LogGroupNames": ["/myapp/api"],
    "LogFormat": "JSON",
    "Fields": {
      "1": "$.userId",
      "2": "$.statusCode"
    },
    "Contribution": {
      "Keys": ["$.userId"],
      "ValueOf": "$.userId",
      "Filters": [{"Match": "$.statusCode", "EqualTo": 500}]
    },
    "AggregateOn": "Count"
  }'
```

```
Built-in Contributor Insights rules for AWS services:
├── DynamoDB: ConsumedReadCapacityUnits by TableName/Key
├── API Gateway: 4xx/5xx errors by route
└── VPC Flow Logs: top talkers by source IP

Cost: $0.02 per rule per hour + $0.01 per million log events matched
```

💬 Short Crisp Interview Answer

"CloudWatch Dashboards provide visual, real-time monitoring combining metrics from multiple services, regions, and accounts. Metric Math computes derived metrics like error rate percentage (5xx/total × 100) and anomaly detection learns normal behavior to alert on deviations rather than fixed thresholds — much better for metrics with time-of-day or seasonal patterns, though it needs two weeks of data to learn patterns reliably. Contributor Insights solves 'who is causing this problem?' — it automatically ranks top contributors from log data in real time, so when error rates spike you can immediately see which user IDs, IP addresses, or DynamoDB partition keys are responsible, without writing custom aggregation code."

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
Dashboard cost	$3/month each (first 3 free). Budget for dashboards in production accounts
Metric Math period alignment	All metrics in an expression must use same period or explicitly specify each
Anomaly detection training period	Needs 2 weeks of data for reliable seasonal pattern detection
Contributor Insights log format	Must specify log format (JSON/CLF/ELF) matching your actual log lines exactly

---

7.5 AWS X-Ray — Tracing, Segments, Sampling

🟢 What It Is in Simple Terms

X-Ray is AWS's distributed tracing service. When a user request flows through multiple services (API Gateway → Lambda → DynamoDB → SQS → another Lambda), X-Ray tracks that entire journey as a single trace, so you can see exactly where latency is coming from and where errors occur in the call chain.

🔍 Why It Exists / What Problem It Solves

```
Problem: User reports "my request is slow." You have:
├── 1 API Gateway
├── 3 Lambda functions
├── 2 DynamoDB tables
├── 1 SQS queue
└── 1 external HTTP call to a payment provider

Without tracing:
→ Check Lambda logs... find request ID... search each function...
→ Check DynamoDB metrics... which table? which partition key?
→ 30+ minutes of log archaeology to find the bottleneck

With X-Ray:
→ Click on the slow trace
→ See the waterfall:
  API GW (5ms) → Lambda 1 (200ms) → DynamoDB (180ms ← HERE)
  → Lambda 2 (10ms)
→ 30 seconds to find the problem
```

⚙️ How X-Ray Works Internally

```
X-Ray Trace Architecture:

User Request
     │
     ▼
API Gateway ─── Trace Header: X-Amzn-Trace-Id: Root=1-abc123; Sampled=1
     │           (propagates this header to ALL downstream calls)
     ▼
Lambda Function 1
├── Segment: "my-lambda-1"  (this function's total work)
│   ├── Subsegment: "DynamoDB GetItem"    (3ms)
│   ├── Subsegment: "SQS SendMessage"     (8ms)
│   └── Annotation: {userId: "user123"}
│
     │ invokes
     ▼
Lambda Function 2
└── Segment: "my-lambda-2"
    └── Subsegment: "External HTTP: stripe.com"  (450ms ← slow!)

X-Ray Service Map (visual):
API GW → Lambda 1 → DynamoDB
                  ↘ SQS → Lambda 2 → stripe.com (slow!)

Automatically shows:
├── Each service as a node in the graph
├── Latency distribution per node
```

```
├── Error / fault / throttle rates per edge
└── Click any node to see individual traces
```

🧩 Core Concepts

```
Trace:
├── End-to-end record of a single request journey
├── Identified by unique Trace ID propagated via HTTP header
└── Contains all segments from all services that handled the request

Segment:
├── Work done by ONE service (one Lambda, one EC2 app instance)
├── Has: start time, end time, HTTP info, metadata, annotations
└── Can contain multiple subsegments

Subsegment:
├── Work done WITHIN a segment (a DynamoDB call, an HTTP request)
├── AWS SDK automatically creates subsegments for AWS service calls
└── Custom subsegments for your own code blocks

Annotations:
├── Indexed key-value pairs (searchable in X-Ray console + API)
├── Use for: user ID, order ID, tenant ID, feature flags
├── Can filter traces by annotation value
└── Max: 50 annotations per trace

Metadata:
├── Non-indexed data (visible in trace detail, not searchable)
├── Use for: large debug payloads, full request/response bodies
└── Max: 32 KB per segment

HTTP Header propagation:
X-Amzn-Trace-Id: Root=1-58406520-a006649127e371903a2de979;Sampled=1
├── Root:    Trace ID (32 hex chars)
├── Parent:  Parent segment ID
└── Sampled: 1=sampled, 0=not sampled, ?=no sampling decision yet

Each service reads this header and:
1. Creates a segment with this Trace ID
2. Adds its own Segment ID as the new Parent
3. Passes the modified header to all downstream calls
```

🧩 Sampling

```
Problem: High-traffic services have millions of requests per minute.
         Tracing every single one = expensive + overwhelming data.
         Solution: Trace only a SAMPLE of requests.
```

```
Default sampling rule:
├── 1 request per second (reservoir) — always traced regardless
└── 5% of additional requests beyond the reservoir

Custom sampling rules (priority order, lower = higher priority):
├── Fixed rate:  sample X% of matching requests
├── Reservoir:   always sample N per second minimum
└── Match on:    service name, HTTP method, URL path, host
```

```
aws xray create-sampling-rule \
  --sampling-rule '{
    "RuleName": "ProductionAPI",
    "Priority": 1,
    "FixedRate": 0.05,
    "ReservoirSize": 10,
    "ServiceName": "prod-api",
    "ServiceType": "AWS::Lambda::Function",
    "Host": "*",
    "HTTPMethod": "*",
    "URLPath": "*",
    "ResourceARN": "*"
  }'
```

```
Sampling strategies by use case:
├── Critical errors:  100% sampling for 5xx responses
├── Health checks:    0% sampling (not useful to trace pings)
├── Slow endpoints:   higher sampling rate for P99 accuracy
└── Batch jobs:       1% sampling (high volume, low investigation value)

⚠ Sampling happens at the FIRST service in the trace chain.
   If a request is NOT sampled at API Gateway, Lambda won't
   sample it either — the Sampled=0 header propagates downstream.
   This ensures COMPLETE traces (not partial ones).
```

🧩 X-Ray SDK Integration

```python
# Python Lambda with X-Ray tracing
from aws_xray_sdk.core import xray_recorder, patch_all

# Automatically trace all AWS SDK calls (DynamoDB, SQS, S3, etc.)
patch_all()

@xray_recorder.capture('process_order')  # creates a named subsegment
def process_order(order_id):
    # Add annotations (indexed, searchable)
```

```
    xray_recorder.current_subsegment().put_annotation('orderId', order_id)

    # Add metadata (not indexed, for debugging)
    xray_recorder.current_subsegment().put_metadata('orderData',
order_data)

    # These AWS SDK calls automatically appear as subsegments
    table.get_item(Key={'orderId': order_id})
    queue.send_message(MessageBody='...')

def handler(event, context):
    with xray_recorder.in_segment('lambda-order-processor') as segment:
        segment.put_annotation('userId', event['userId'])
        return process_order(event['orderId'])
```

```
# Enable Active Tracing on Lambda (no code changes needed for basic
traces)
aws lambda update-function-configuration \
  --function-name my-function \
  --tracing-config Mode=Active
# Lambda automatically creates root segment
# AWS SDK calls in function automatically become subsegments
```

💬 Short Crisp Interview Answer

"X-Ray is AWS's distributed tracing service. Each request gets a Trace ID that propagates through all services via the X-Amzn-Trace-Id HTTP header. Each service creates a segment with its timing, and AWS SDK calls automatically create subsegments. This gives you a waterfall view of an entire request across services — you can immediately see which service or downstream call is causing latency or errors. Sampling controls cost — the default samples 1 request per second plus 5% of others. You can add annotations (indexed, searchable — great for user IDs and order IDs) and metadata (non-indexed debug data). The X-Ray Service Map automatically visualizes your architecture with latency and error rates on each connection."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Sampling at first service	If not sampled at entry, no downstream services trace it — consistent but means gaps in coverage
Active vs PassThrough mode	Lambda PassThrough: respects upstream header. Active: traces even without upstream header
X-Ray daemon required	On EC2/ECS, must run X-Ray daemon sidecar to receive segments and batch-send to X-Ray API

Gotcha	Detail
Trace header case-sensitivity	AWS SDK handles this automatically — custom HTTP clients must propagate exactly
30-day retention	Traces stored for 30 days only — no long-term historical trend analysis

---

7.6 CloudWatch Embedded Metric Format (EMF)

🟢 What It Is in Simple Terms

EMF lets you emit CloudWatch Metrics directly inside your application's log lines. Instead of calling the CloudWatch Metrics API separately, you write a specially structured JSON log line and CloudWatch automatically extracts the metrics from it. Zero separate API calls — logs and metrics in one single write operation.

🔍 Why It Exists / What Problem It Solves

```
Without EMF:
For every metric from Lambda:
1. Call cloudwatch.put_metric_data() → separate API call (costs money)
2. 10 metrics × 1M invocations/day = 10M PutMetricData calls/day
   = $30+/day just in API call costs

With EMF:
1. Write one structured JSON log line (already writing logs anyway)
2. CloudWatch asynchronously extracts metrics from the log line
3. Zero additional API calls = drastically lower cost

Additional benefits:
├── Metrics extracted asynchronously (zero added Lambda duration)
├── Logs and metrics correlated by request (same log line!)
└── Works with ANY language (just write JSON to stdout)
```

⚙️ EMF Format

```
EMF = specially structured JSON blob written to STDOUT or a log file.
CloudWatch Logs detects the _aws key and extracts metrics automatically.

Minimal EMF structure:
{
  "_aws": {
    "Timestamp": 1705329000000,           ← Unix ms timestamp
```

```
    "CloudWatchMetrics": [{
      "Namespace": "MyApp/Orders",
      "Dimensions": [["Environment", "Service"]],
      "Metrics": [
        {"Name": "OrdersProcessed",      "Unit": "Count"},
        {"Name": "ProcessingLatency",    "Unit": "Milliseconds"}
      ]
    }]
  },
  "Environment":        "prod",           ← dimension value
  "Service":            "order-processor",← dimension value
  "OrdersProcessed":    1,                ← metric value
  "ProcessingLatency":  145,              ← metric value
  "requestId":          "abc123",         ← just a log field (not metric)
  "userId":             "user456"         ← just a log field (not metric)
}

CloudWatch extracts from the above:
├── Metric: MyApp/Orders/OrdersProcessed {Environment=prod, Service=order-
processor} = 1
└── Metric: MyApp/Orders/ProcessingLatency {Environment=prod,
Service=order-processor} = 145

The log line is ALSO stored as a searchable log event in CloudWatch Logs.
```

🧩 EMF Libraries

```python
# Python — using aws-embedded-metrics library
from aws_embedded_metrics import metric_scope

@metric_scope
def handler(event, context, metrics):
    # Set dimensions (low cardinality)
    metrics.set_dimensions({"Service": "order-processor", "Environment":
"prod"})

    # Set namespace
    metrics.set_namespace("MyApp/Orders")

    # Record metric values
    metrics.put_metric("OrdersProcessed",   1,   "Count")
    metrics.put_metric("ProcessingLatency", 145, "Milliseconds")

    # Add log properties (not dimensions — high cardinality is OK here)
    metrics.set_property("requestId", context.aws_request_id)
    metrics.set_property("userId",    event.get("userId"))

    return process_order(event)
    # Decorator automatically flushes EMF JSON to stdout on return
```

```
// Node.js — using aws-embedded-metrics
const { createMetricsLogger, Unit } = require('aws-embedded-metrics');

exports.handler = async (event, context) => {
    const metrics = createMetricsLogger();

    metrics.setNamespace('MyApp/Orders');
    metrics.setDimensions({ Service: 'order-processor', Environment:
'prod' });

    metrics.putMetric('OrdersProcessed',   1,   Unit.Count);
    metrics.putMetric('ProcessingLatency', 145, Unit.Milliseconds);
    metrics.setProperty('requestId', context.awsRequestId);

    await metrics.flush();   // Must flush before Lambda exits!
    return processOrder(event);
};
```

🧩 EMF High-Cardinality Pattern

```
Problem with high-cardinality CloudWatch dimensions:
{"Name": "RequestLatency", dimensions: [{userId: "user123"}]}
→ Every unique userId = a new CloudWatch metric
→ $0.30/month × 100K users = $30,000/month!

EMF solution — use low-cardinality dimensions, high-cardinality
properties:
{
  "_aws": {
    "CloudWatchMetrics": [{
      "Namespace": "MyApp/API",
      "Dimensions": [["Environment"]],    ← LOW cardinality (always
prod/staging)
      "Metrics": [{"Name": "Latency", "Unit": "Milliseconds"}]
    }]
  },
  "Environment":  "prod",
  "Latency":      145,
  "userId":       "user123",   ← log PROPERTY (not dimension — no metric
cost)
  "orderId":      "ord456"     ← log PROPERTY (not dimension — no metric
cost)
}

Result:
├── CloudWatch metric: MyApp/API/Latency {Environment=prod} = 145
│   (one cheap metric with low-cardinality dimension)
└── Log line contains userId and orderId
    → Logs Insights query:
```

```
        filter userId = "user123" | stats avg(Latency) by bin(5m)
      → Get per-user latency from LOGS without per-user METRICS

  This is the EMF superpower: structured logs that are ALSO metrics.
  Use the cheap metric for dashboards and alarms.
  Use the logs for per-request, per-user debugging.
```

💬 Short Crisp Interview Answer

"EMF lets you publish CloudWatch metrics by writing structured JSON to your application logs — no separate PutMetricData API calls. The special `_aws` key in the JSON tells CloudWatch Logs to extract metrics asynchronously. This dramatically reduces costs (zero separate API calls), adds no latency (writing logs is already happening), and keeps log context and metrics correlated on the same line. The power pattern is using low-cardinality fields as dimensions (Environment, Service) but logging high-cardinality data like userId and orderId as log properties — then querying metrics for dashboards and Logs Insights for per-request debugging. This gives you the best of both worlds without paying per-user metric costs."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Flush required	Must call metrics.flush() before Lambda returns — metrics are lost if not flushed
Timestamp precision	EMF uses millisecond epoch timestamps. Wrong timestamp = metric in wrong time bucket
Dimension cardinality	Each unique dimension combination = new metric = $0.30/month. Keep dimensions low-cardinality
Malformed `_aws` block	Any invalid JSON in the `_aws` block → CloudWatch silently drops the entire metric
Lambda log buffering	Use the library decorator to guarantee flush is called before function exits

- **Silent metric loss with no error is the most dangerous EMF pitfall** — always use the `@metric_scope` decorator or ensure `await metrics.flush()` is called in every code path including exception handlers. A Lambda that exits without flushing drops all metrics with no warning whatsoever.
- **Malformed `_aws` block silently drops the entire metric** — any invalid JSON in the `_aws` structure causes CloudWatch to ignore the metric extraction entirely while still storing the log line. Always validate EMF output in staging before production deployment.

7.7 CloudWatch Agent — Custom Metrics, Procstat

🟢 What It Is in Simple Terms

The CloudWatch Agent is a software daemon you install on EC2 instances or container hosts. It collects OS-level metrics (memory, disk, processes) that AWS doesn't collect by default, tails log files, and ships everything to CloudWatch. Without it, you're operating blind on memory usage and disk space.

🔍 Why It Exists / What Problem It Solves

```
What EC2 provides BY DEFAULT in CloudWatch (hypervisor-visible only):
├── CPUUtilization
├── NetworkIn / NetworkOut
├── DiskReadOps / DiskWriteOps (instance store only)
└── StatusCheckFailed

What EC2 does NOT provide by default (OS-level — invisible to hypervisor):
├── Memory utilization (hypervisor cannot see inside the OS)
├── Disk space used / available (not disk I/O — actual free space)
├── Swap utilization
├── Process-level metrics (which process uses the most CPU or RAM)
└── Custom application log files on disk

CloudWatch Agent provides ALL of the above.
```

🧩 CloudWatch Agent Configuration

```
{
  "metrics": {
    "append_dimensions": {
      "AutoScalingGroupName": "${aws:AutoScalingGroupName}",
      "ImageId":              "${aws:ImageId}",
      "InstanceId":           "${aws:InstanceId}",
      "InstanceType":         "${aws:InstanceType}"
    },
    "metrics_collected": {
      "mem": {
        "measurement": [
          "mem_used_percent",
          "mem_available_percent"
        ],
        "metrics_collection_interval": 60
      },
      "disk": {
        "measurement": [
          "disk_used_percent",
          "disk_free"
        ],
        "resources": ["/", "/data"],
```

```
          "metrics_collection_interval": 60
        },
        "swap": {
          "measurement": ["swap_used_percent"],
          "metrics_collection_interval": 60
        },
        "cpu": {
          "measurement": [
            "cpu_usage_idle",
            "cpu_usage_user",
            "cpu_usage_system"
          ],
          "totalcpu": true,
          "metrics_collection_interval": 30
        },
        "procstat": [
          {
            "pattern": "nginx",
            "measurement": [
              "cpu_usage",
              "memory_rss",
              "num_threads",
              "read_bytes",
              "write_bytes"
            ]
          },
          {
            "pid_file": "/var/run/myapp.pid",
            "measurement": ["cpu_usage", "memory_rss"]
          }
        ]
      }
    },
    "logs": {
      "logs_collected": {
        "files": {
          "collect_list": [
            {
              "file_path":        "/var/log/nginx/access.log",
              "log_group_name":   "/ec2/prod/nginx/access",
              "log_stream_name":  "{instance_id}",
              "timestamp_format": "%d/%b/%Y:%H:%M:%S %z"
            },
            {
              "file_path":                "/var/log/myapp/application.log",
              "log_group_name":           "/ec2/prod/myapp",
              "log_stream_name":          "{instance_id}-{hostname}",
              "multi_line_start_pattern": "^\\d{4}-\\d{2}-\\d{2}"
            }
          ]
        }
      }
    }
  }
```

🧩 Procstat Plugin

```
Procstat = per-process statistics collection

Why it matters:
├── Identify which specific process is consuming memory
├── Monitor critical service processes (nginx, postgres, myapp)
├── Alert on memory leaks (memory_rss growing over time)
└── Debug CPU contention (which process caused the spike)

Process selection methods:
├── pattern:  regex match on process name or cmdline string
├── pid_file: read PID from /var/run/process.pid
├── pid:      specific numeric PID (static — usually not useful)
└── exe:      exact executable name (e.g., "postgres")

Key procstat metrics:
├── cpu_usage:          % CPU this process is using
├── memory_rss:         resident set size (actual RAM in use)
├── memory_vms:         virtual memory size
├── num_threads:        current thread count
├── num_fds:            open file descriptors
├── read_bytes:         disk reads per second for this process
├── write_bytes:        disk writes per second for this process
└── rlimit_memory_rss_hard: the OOM kill threshold

⚠ Procstat metrics appear in CWAgent namespace:
   Namespace:  CWAgent
   MetricName: procstat_memory_rss
   Dimensions: {process_name, InstanceId, AutoScalingGroupName}
```

🧩 Deploying CloudWatch Agent at Scale

```bash
# Method 1: SSM Parameter Store (recommended for fleets)
# Step 1: Store config in SSM
aws ssm put-parameter \
  --name "/cloudwatch-agent/config/prod" \
  --type "String" \
  --value file://cloudwatch-agent-config.json

# Step 2: Distribute to all instances via SSM Run Command
aws ssm send-command \
  --instance-ids i-0abc123 \
  --document-name "AmazonCloudWatch-ManageAgent" \
  --parameters '{"action": ["configure"], "mode": ["ec2"],
    "optionalConfigurationLocation": ["/cloudwatch-agent/config/prod"]}'
```

```
Method 2: Bake into AMI:
1. Install agent during AMI build
2. Copy config file into AMI at build time
3. All instances launched from AMI have agent pre-configured
Best for: immutable infrastructure

Method 3: EC2 User Data:
Bootstrap agent installation + configuration at first launch.
Best for: auto-scaling groups where instances come and go

Required IAM permissions for EC2 instance role:
├── cloudwatch:PutMetricData
├── logs:PutLogEvents
├── logs:CreateLogGroup
├── logs:CreateLogStream
└── ssm:GetParameter (for SSM config method)
```

💬 Short Crisp Interview Answer

"The CloudWatch Agent is a daemon installed on EC2 instances that collects metrics AWS can't see from the hypervisor — primarily memory utilization, disk space, and swap. Without it, you're operating EC2 blindly on these critical signals. Configuration is a JSON file specifying which metrics to collect, at what intervals, and which log files to tail. The procstat plugin extends this to process-level monitoring — you can track memory_rss per process by name or PID file, enabling memory leak detection and CPU attribution to specific processes. In production, deploy the config via SSM Parameter Store so you can update agent configuration across hundreds of instances without SSH access."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Memory not in default EC2 metrics	EC2 CPUUtilization is free and automatic. Memory requires CloudWatch Agent
Agent IAM permissions	Instance role must allow cloudwatch:PutMetricData, logs:PutLogEvents, ssm:GetParameter
Procstat process not found	If pattern doesn't match any running process, no metrics emitted — no warning
Multi-line log handling	multi_line_start_pattern must match your log format or multi-line events are split incorrectly
High-resolution agent metrics cost	Collecting every 1 second — at $0.30/metric/month, many metrics becomes expensive fast

---

7.8 AWS Distro for OpenTelemetry (ADOT)

🟢 What It Is in Simple Terms

ADOT is AWS's distribution of OpenTelemetry — the open-source observability standard. It lets you collect metrics, traces, and logs using vendor-neutral instrumentation, then send that data to AWS services (CloudWatch, X-Ray) AND/OR third-party tools (Datadog, Prometheus, Grafana) simultaneously, without vendor lock-in.

🔍 Why It Exists / What Problem It Solves

```
Without OpenTelemetry:
├── Instrument for X-Ray  → tied to X-Ray SDK  → switching to Datadog =
re-instrument everything
├── Instrument for Datadog → tied to Datadog agent → switching = re-
instrument everything
└── Each vendor has different APIs, different concepts, different SDKs

With OpenTelemetry:
├── Instrument ONCE using standard OTel SDK (vendor-neutral)
├── Send to X-Ray today
├── Add Prometheus endpoint tomorrow (same instrumentation)
├── Switch to Grafana Cloud next quarter
└── Never re-instrument your application code
```

⚙️ ADOT Architecture

```
ADOT Architecture:

Your Application (Python / Java / Node / Go)
├── OpenTelemetry SDK (traces, metrics, logs)
└── Exports via OTLP protocol to ADOT Collector

ADOT Collector (sidecar or DaemonSet):
├── Receivers:  OTLP, Prometheus, StatsD, Jaeger, Zipkin
├── Processors: batch, filter, transform, resource detection
└── Exporters:
    ├── AWS X-Ray     → distributed tracing (converted to X-Ray format)
    ├── CloudWatch    → metrics (via EMF format)
    ├── Prometheus    → metrics scraping endpoint
    ├── Datadog       → metrics + traces
    └── OTLP backend  → any OpenTelemetry-compatible backend

ADOT Collector as ECS Sidecar:
Task Definition:
```

```
├── Container: my-app
│   └── OTLP export to: http://localhost:4317
└── Container: aws-otel-collector
    ├── Receives OTLP from my-app on localhost
    └── Exports to X-Ray + CloudWatch metrics

ADOT Collector on EKS (two deployment patterns):
├── DaemonSet: one collector per node (shared by all pods on node)
└── Sidecar:   one collector per pod (more isolation, more resource cost)
```

🧩 ADOT Collector Configuration

```yaml
# otel-config.yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

  prometheus:
    config:
      scrape_configs:
        - job_name: 'my-app'
          static_configs:
            - targets: ['localhost:8080']

processors:
  batch:
    timeout: 1s
    send_batch_size: 50

  resourcedetection:
    detectors: [ec2, ecs, eks]
    # Automatically adds: InstanceId, Region, ClusterName, etc.

exporters:
  awsxray:
    region: us-east-1

  awsemf:
    region: us-east-1
    namespace: MyApp/Service
    dimension_rollup_option: "NoDimensionRollup"
    metric_declarations:
      - dimensions: [["Service", "Environment"]]
        metric_name_selectors:
          - "^http.server.*"
```

```
service:
  pipelines:
    traces:
      receivers:  [otlp]
      processors: [batch, resourcedetection]
      exporters:  [awsxray]
    metrics:
      receivers:  [otlp, prometheus]
      processors: [batch, resourcedetection]
      exporters:  [awsemf]
```

🧩 ADOT on Lambda — Zero-Code Instrumentation

```
# Lambda with ADOT — zero application code changes needed
# Step 1: Add Lambda Layer:
#   arn:aws:lambda:us-east-1::layer:AWSOpenTelemetryDistro:*
# Step 2: Set environment variable:
#   AWS_LAMBDA_EXEC_WRAPPER=/opt/otel-handler
#
# Lambda wrapper automatically instruments:
# ├── Function invocation (creates root trace span)
# ├── AWS SDK calls (DynamoDB, S3, SQS, SNS)
# └── HTTP client calls (requests, axios, etc.)

# For CUSTOM instrumentation within your function:
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

tracer = trace.get_tracer(__name__)

def handler(event, context):
    with tracer.start_as_current_span("process-order") as span:
        span.set_attribute("order.id",  event["orderId"])
        span.set_attribute("user.id",   event["userId"])

        try:
            result = process_order(event)
            span.set_status(Status(StatusCode.OK))
            return result
        except Exception as e:
            span.set_status(Status(StatusCode.ERROR, str(e)))
            raise
```

🧩 ADOT vs X-Ray SDK vs CloudWatch Agent

```
┌──────────────────────┬──────────────────┬──────────────┬──────────────┐
│ Feature              │ ADOT             │ X-Ray SDK    │ CW Agent     │
```

```
  ├──────────────────────┼──────────────────┼────────────────┼────────────────┤
  │ Tracing              │ ✅ OTel spans    │ ✅ Segments    │ ❌             │
  │ Metrics              │ ✅ OTel metrics  │ ❌             │ ✅ OS metrics  │
  │ Logs                 │ ✅ (limited)     │ ❌             │ ✅ File tails  │
  │ Vendor-neutral       │ ✅ Yes           │ ❌ AWS only    │ ❌ AWS only    │
  │ Multi-destination    │ ✅ Yes           │ ❌             │ ❌             │
  │ Complexity           │ Higher           │ Lower          │ Medium         │
  └──────────────────────┴──────────────────┴────────────────┴────────────────┘

When to use ADOT:
├── Already using OpenTelemetry in your codebase
├── Want to send data to multiple backends simultaneously
├── Building for portability (may move away from AWS someday)
└── Large engineering org with standardized OTel tooling

When to use X-Ray SDK:
├── Pure AWS environment, no multi-cloud plans
├── Simple distributed tracing is all you need
└── Team not familiar with OpenTelemetry concepts

When to use both:
└── ADOT for application tracing → X-Ray
    CloudWatch Agent for OS metrics → CloudWatch
    (they complement each other, not replace each other)
```

💬 Short Crisp Interview Answer

"ADOT is AWS's distribution of OpenTelemetry — the vendor-neutral observability standard. It lets you instrument once using OpenTelemetry SDKs and ship telemetry to multiple backends simultaneously — X-Ray for traces, CloudWatch via EMF for metrics, Prometheus for scraping, and any other OTel-compatible backend. The ADOT Collector is a sidecar or daemon that receives OTLP data from your app, processes it, and exports to configured backends. The key benefit is avoiding vendor lock-in — if you later need to add Datadog or move to Grafana Cloud, you change the Collector configuration, not your application instrumentation code. For pure AWS-only environments where portability isn't a concern, the X-Ray SDK is simpler."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
OTel and X-Ray span format	OTel trace IDs must be converted to X-Ray format — ADOT Collector handles this automatically
Collector resource usage	ADOT Collector sidecar uses CPU and memory — account for this in ECS/EKS task limits
ADOT Lambda layer ARN	Layer ARN is region-specific — check AWS documentation for correct ARN per region

Gotcha	Detail
Prometheus receiver pull model	Collector must reach app's Prometheus endpoint — in EKS, use pod annotations for auto-discovery

---

7.9 Container Insights & Lambda Insights

🟢 What It Is in Simple Terms

Container Insights and Lambda Insights are enhanced observability solutions that give you deep, automatic metrics and log collection for containers and Lambda functions — far beyond the basic metrics you get out of the box. They make the invisible visible: per-container resource usage, cold start rates, Lambda memory efficiency, and more.

🧩 Container Insights

```
Container Insights collects:

For ECS:
├── Cluster level:   CPU/memory utilization, task counts
├── Service level:   running tasks, pending tasks, CPU/memory per service
├── Task level:      per-task CPU, memory, network I/O
└── Container level: per-container CPU and memory within a task

For EKS:
├── Cluster level:   node count, running pod count
├── Node level:      CPU/memory per node, network, disk
├── Pod level:       CPU/memory per pod, restart counts
└── Container level: per-container CPU/memory within a pod

Metrics go into: CloudWatch namespace ContainerInsights
Logs go into:    /aws/ecs/containerinsights/* or
                 /aws/eks/containerinsights/*
```

```bash
# Enable Container Insights on ECS cluster
aws ecs update-cluster-settings \
  --cluster prod-cluster \
  --settings name=containerInsights,value=enabled

# Enable Container Insights on EKS via CloudWatch add-on
aws eks create-addon \
  --cluster-name prod-cluster \
  --addon-name amazon-cloudwatch-observability \
  --service-account-role-arn arn:aws:iam::123:role/cw-agent-role
```

🧩 Container Insights Architecture

```
Container Insights Data Flow on EKS:

┌──────────────────────────────────────────────────────────────┐
│  EKS Node                                                    │
│                                                              │
│  ┌──────────────────┐   ┌──────────────────────────┐        │
│  │ CloudWatch Agent  │   │ Fluent Bit               │        │
│  │ (DaemonSet)       │   │ (DaemonSet)              │        │
│  │                   │   │                          │        │
│  │ Collects:         │   │ Collects:                │        │
│  │ - cAdvisor data   │   │ - Container stdout/stderr│        │
│  │ - Kubelet metrics │   │ - Node OS logs           │        │
│  │ - Node metrics    │   │                          │        │
│  └──────────────────┘   └──────────────────────────┘        │
└──────────────┬──────────────────────────┬──────────────────┘
               │ EMF format               │ Log events
               ▼                          ▼
      CloudWatch Metrics         CloudWatch Logs
      (ContainerInsights)        (/aws/eks/cluster-name/*)

Pre-built Container Insights dashboards:
├── ECS Clusters:  cluster-level health overview
├── ECS Services:  service task counts and resource usage
├── ECS Tasks:     per-task container breakdown
├── Pod Performance: CPU, memory, restarts per pod (EKS)
└── Node Performance: node CPU, memory, disk (EKS)
```

🧩 Lambda Insights

```
Standard Lambda metrics (AWS/Lambda namespace — free, automatic):
├── Invocations
├── Duration
├── Errors
├── Throttles
└── ConcurrentExecutions

Lambda Insights ADDITIONAL metrics (LambdaInsights namespace):
├── init_duration:      cold start initialization time per invocation
├── memory_utilization: actual memory used vs configured limit (%)
├── total_memory:       configured memory limit in MB
├── used_memory_max:    peak memory usage during the invocation
├── rx_bytes:           network bytes received per invocation
├── tx_bytes:           network bytes transmitted per invocation
└── total_network:      total network bytes per invocation
```

```
Why these metrics matter:
init_duration:       identifies cold start frequency and severity
memory_utilization:  find over-provisioned functions → reduce cost
                     find under-provisioned functions → prevent OOM kills

Example actionable insight:
Function configured for 1024MB, peak usage is 150MB
→ Right-size to 256MB
→ 75% memory cost reduction + same CPU performance (or better)
```

```
# Enable Lambda Insights by adding the Extension Layer
aws lambda update-function-configuration \
  --function-name my-function \
  --layers arn:aws:lambda:us-east-
1:580247275435:layer:LambdaInsightsExtension:38
```

```
Lambda Insights mechanism:
├── Lambda Extension runs alongside your function code
├── Collects enhanced metrics from the Lambda runtime environment
├── Emits via EMF format to /aws/lambda/insights log group
└── CloudWatch automatically extracts metrics from the EMF

Lambda Insights log line (EMF format):
{
  "_aws": {
    "Timestamp": 1705329000000,
    "CloudWatchMetrics": [{
      "Namespace": "LambdaInsights",
      "Dimensions": [["function_name", "resource"]],
      "Metrics": [
        {"Name": "memory_utilization", "Unit": "Percent"},
        {"Name": "init_duration",      "Unit": "Milliseconds"}
      ]
    }]
  },
  "function_name":    "my-function",
  "resource":         "my-function:$LATEST",
  "memory_utilization": 14.6,
  "init_duration":      312,
  "used_memory_max":    150
}
```

🧩 Container Insights vs Lambda Insights

```
┌──────────────────────────┬──────────────────┬──────────────────┐
│ Feature                  │ Container Insights│ Lambda Insights  │
```

```
├──────────────────────────────┼──────────────────┼──────────────────┤
│ Target                       │ ECS, EKS nodes   │ Lambda functions │
│ Cold start visibility        │ N/A              │ ✅ init_duration  │
│ Memory utilization           │ ✅ Per container  │ ✅ Per invocation │
│ Log collection               │ Fluent Bit       │ /aws/lambda/insights│
│ Auto dashboards              │ ✅ Yes            │ ✅ Yes            │
│ Deployment                   │ DaemonSet / addon│ Layer ARN added  │
└──────────────────────────────┴──────────────────┴──────────────────┘
```

🏭 Real World Production Observability Stack

```
Full observability for a production EKS microservice:

EKS Service (order-processor):
├── Container Insights (DaemonSet):
│   Metrics: pod CPU, memory, network → ContainerInsights namespace
│
├── Fluent Bit (DaemonSet):
│   Logs: container stdout → /aws/eks/prod-cluster/order-processor
│
├── ADOT Collector (sidecar per pod):
│   Traces → X-Ray (waterfall, service map)
│   Metrics → CloudWatch (EMF, custom business metrics)
│
├── Application (order-processor):
│   EMF: OrdersProcessed, OrderLatency, OrderFailures (per request)
│   OTel SDK: spans for DynamoDB calls, external HTTP calls
│
└── Dashboards:
    Dashboard 1: Service Health (error rate, P99 latency, throughput)
    Dashboard 2: Infrastructure (CPU, memory, pod count per node)
    Dashboard 3: Business (orders/hour, failure rate by reason code)

Alarms:
├── Composite: P99 latency > 500ms AND error rate > 1% → page on-call
├── Pod memory > 80% → scale out the deployment
├── Pod restart count > 3 in 5 minutes → page on-call
└── Lambda cold start rate > 20% → investigate Provisioned Concurrency

Incident workflow:
1. Composite alarm fires → P99 latency degraded
2. Dashboard → which endpoint is slow?
3. Contributor Insights → which user ID generating most errors?
4. X-Ray Service Map → which service in the call chain is slow?
5. X-Ray trace detail → which DynamoDB table? which partition key?
6. Logs Insights → correlated log lines from the slow requests
7. EMF metrics → OrdersProcessed drop confirms business impact
```

💬 Short Crisp Interview Answer

"Container Insights provides deep, automatic metrics for ECS and EKS — from cluster level down to individual containers. It uses the CloudWatch Agent as a DaemonSet to collect cAdvisor and Kubelet metrics in EMF format, and Fluent Bit for log collection. Pre-built dashboards are generated automatically. Lambda Insights is the Lambda equivalent — a Lambda Extension layer that collects enhanced per-invocation metrics like init_duration for cold start detection, memory_utilization for right-sizing, and network I/O. The most actionable Lambda Insight is memory_utilization — if your function uses 14% of configured memory, you're overpaying roughly 7x on memory and its associated CPU allocation. The fix is simply right-sizing the function to match actual usage."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Container Insights costs	Enabling it generates many metrics → significant cost at scale. Review before enabling on large clusters
Lambda Insights layer ARN	ARN is region-specific. A different ARN is needed per AWS region — check AWS docs
Fluent Bit vs CW Agent for logs	Fluent Bit preferred for Kubernetes log collection (lighter, better Kubernetes-native)
Container Insights memory	Shows utilization against configured limit — if no limits set, metrics may be misleading
Lambda Insights extension overhead	Extension adds ~10-15ms to Lambda init time — usually acceptable, but notable for very cold start-sensitive functions

---

🔗 Category 7 — Full Connections Map

```
OBSERVABILITY connects to:

CloudWatch Metrics
├── EC2, RDS, Lambda, ECS, ALB... → emit metrics automatically
├── CloudWatch Alarms    → threshold-based alerting on metrics
├── CloudWatch Dashboards→ visualize and combine metrics
├── Auto Scaling         → scale based on CloudWatch alarm triggers
├── Metric Filters       → create metrics FROM CloudWatch Logs content
└── EMF                  → metrics embedded directly in log lines

CloudWatch Logs
├── Lambda, ECS, EC2     → all ship application logs here
├── CloudWatch Agent     → tails OS log files, ships to CW Logs
├── Fluent Bit           → ships Kubernetes container logs to CW Logs
├── Subscription Filters → stream to Kinesis, Lambda, Kinesis Firehose
├── Logs Insights        → interactive query language on log data
```

```
  ├── Metric Filters       → extract metric values from log content
  └── X-Ray                → traces available as enriched structured logs

  X-Ray
  ├── Lambda      → Active Tracing mode (no code changes needed)
  ├── ECS / EKS   → via X-Ray daemon sidecar or ADOT Collector
  ├── API Gateway → emits traces to X-Ray natively
  ├── SNS / SQS / DynamoDB → traced automatically by X-Ray SDK
  └── ADOT        → converts OTel spans to X-Ray format and exports

  ADOT
  ├── X-Ray          → trace export (with format conversion)
  ├── CloudWatch     → metric export via EMF
  ├── Prometheus     → metric scrape and export
  ├── Datadog        → multi-destination export
  ├── Any OTel backend → Grafana, Jaeger, Zipkin, etc.
  └── Lambda, ECS, EKS → instrumentation targets

  Container Insights / Lambda Insights
  ├── CloudWatch Metrics → ContainerInsights / LambdaInsights namespaces
  ├── CloudWatch Logs    → /aws/ecs/containerinsights/*,
  /aws/lambda/insights
  ├── CloudWatch Agent   → DaemonSet on EKS, cluster setting on ECS
  └── Fluent Bit         → container log collection to CloudWatch Logs
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
CloudWatch Metrics namespace	Container for metrics. AWS/EC2, AWS/Lambda, or your custom MyApp/API
Dimensions = metric identity	Different dimension values = completely different metric time series
Standard resolution	1-minute granularity. 1-minute data retained 15 days
High resolution	1-60 second granularity. 1-second data retained only 3 hours
Custom metric cost	$0.30/month per unique namespace + metric name + dimension combination
S3 metrics frequency	BucketSizeBytes updated ONCE per day — not real-time
Log retention default	NEVER EXPIRE — always set a retention policy immediately
Metric filter backfill	Does NOT backfill historical data. Only processes future log events
Logs Insights cost	$0.005 per GB scanned
Logs Insights row limit	10,000 rows max. Use aggregations instead of raw log pulls
Alarm states	OK, ALARM, INSUFFICIENT_DATA
Alarm M of N	datapointsToAlarm/evaluationPeriods. 2/3 prevents flapping noise

Topic	Key Fact
Missing data = breaching	Use for metrics that should always exist (Lambda errors, API requests)
Composite alarm operators	AND, OR, NOT — combine multiple individual alarms logically
Composite alarms no Auto Scaling	Cannot directly trigger ASG scaling — route through SNS → Lambda
X-Ray trace propagation	X-Amzn-Trace-Id header passed between ALL services in the call chain
X-Ray segment	Work done by one single service instance
X-Ray subsegment	Work within a segment (one DynamoDB call, one HTTP call)
X-Ray annotation	Indexed (searchable in console). Max 50 per trace. Use for userId, orderId
X-Ray metadata	Not indexed (not searchable). Use for large debug payloads
X-Ray default sampling	1 request/second guaranteed + 5% of additional requests
Sampling at first service	Not sampled at entry = not sampled downstream — consistent complete traces
EMF _aws key	Triggers CloudWatch metric extraction from the JSON log line
EMF flush required	Must call flush() before Lambda exits or all metrics are silently lost
EMF dimension cardinality	Each unique dimension combination = new metric = $0.30/month
CloudWatch Agent — memory	EC2 memory NOT in default hypervisor metrics. Requires agent
CloudWatch Agent — disk space	Disk space (not I/O ops) NOT in default metrics. Requires agent
Procstat namespace	CWAgent namespace. MetricName: procstat_memory_rss
ADOT benefit	Instrument once, export to multiple backends. No vendor lock-in
ADOT vs X-Ray SDK	ADOT = portability + multi-destination. X-Ray SDK = simpler, AWS-only
Container Insights enable	aws ecs update-cluster-settings OR EKS CloudWatch add-on
Lambda Insights — init_duration	Cold start detection and measurement per invocation
Lambda Insights — memory_utilization	Right-sizing signal. 14% utilization = configured 7x too much memory
Lambda Insights mechanism	Lambda Extension layer emitting EMF to /aws/lambda/insights log group
Contributor Insights use case	Ranks top contributors (user IDs, IPs, partition keys) causing a metric
Anomaly detection training	Needs ~2 weeks of data to learn seasonal and time-of-day patterns

---

Category 7: Observability & Monitoring — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation